

OS Lecture 3 — Processes

Complete Study Guide · Memorize + Understand

1. What Is a Process?

■ MEMORIZE — Definition

- A process = a program in execution (an active entity, not a static file).
- It has: program code (text), data section (global vars), stack (temp data), heap (dynamic alloc), and current state (PC + registers).
- Context = ALL information the OS needs to manage the process.

■ UNDERSTAND — Program vs Process

- A program is passive (just bytes on disk). A process is active — it is the program currently running with its own memory and CPU state.
- The same program can be multiple processes at the same time (e.g. two users running the same editor).

2. Process Memory Layout

Segment	Contents	Grows Toward
Stack (top)	Function params, return addresses, local variables	↓ Downward (toward heap)
Heap	Memory dynamically allocated at runtime (malloc/new)	↑ Upward (toward stack)
Data	Global and static variables (initialized + BSS)	Fixed size
Text (bottom)	Program code — machine instructions	Fixed size (read-only)

■ MEMORIZE — Memory Segment Names

- Text = code | Data = global vars | Heap = dynamic alloc | Stack = local/temp
- Stack grows DOWN, Heap grows UP — they share the middle space.

3. Process Control Block (PCB)

■ UNDERSTAND — What & Why

- The PCB is a data structure created and managed by the OS for every process.
- It contains enough information to interrupt a running process and later resume it — this is what enables multitasking.
- It is the most important data structure in an OS. All OS modules (scheduler, interrupt handler, memory manager) read/modify it.

■ MEMORIZE — PCB Fields (must recall all categories)

- Process Identification: PID (unique ID), parent PID, user ID.
- Process State: running / ready / blocked / new / exit.
- Program Counter: address of next instruction to execute.
- CPU Registers: all register contents (saved on context switch).
- CPU Scheduling Info: priority, scheduling queue pointers.
- Memory Management Info: page/segment table pointers, memory limits.
- Accounting Info: CPU time used, clock time elapsed, time limits.
- I/O Status Info: list of open files, I/O devices allocated.

4. Process State Models

4A · Two-State Model (simplest)

■ MEMORIZE — Two States

- Running — currently executing on the CPU.
- Not-Running — waiting in a queue for its turn.
- Dispatcher moves process from Not-Running → Running; paused process goes back to Not-Running.

4B · Five-State Model (standard exam model)

State	Meaning
New	Process just created; PCB exists but not yet loaded into memory.
Ready	In memory, waiting for CPU — can run as soon as OS gives it the processor.
Running	Currently executing on the CPU.
Blocked/Waiting	Waiting for an event (I/O completion, semaphore) — cannot use CPU yet.
Exit	Process has finished; released from pool of executable processes.

■ MEMORIZE — Five-State Transitions (all 9)

- Null → New: process is created.
- New → Ready: OS admits the process (memory available).
- Ready → Running: dispatcher selects this process.
- Running → Exit: process terminates (calls exit()).
- Running → Ready: time slice expired OR higher-priority process is ready.
- Running → Blocked: process waits for an event (e.g. I/O request).
- Blocked → Ready: the waited-for event has occurred.
- Ready → Exit: parent/OS terminates the process.
- Blocked → Exit: same reasons as above.

5. Suspended Processes (7-State Model)

■ UNDERSTAND — Why Suspend?

- CPU is much faster than I/O — all processes could end up Blocked at the same time, wasting the CPU.
- Solution: swap blocked (or even ready) processes OUT to disk, freeing main memory for other processes.
- Blocked state → Blocked/Suspend when swapped to disk.
- Two new states are added: Blocked/Suspend and Ready/Suspend.

State	Location	Meaning
Ready	Main memory	In RAM, ready to run immediately.
Blocked	Main memory	In RAM, waiting for an event.
Ready/Suspend	Secondary (disk)	Swapped out but ready to run once loaded back.
Blocked/Suspend	Secondary (disk)	Swapped out AND still waiting for an event.

■ MEMORIZE — Key Suspend Transitions

- Blocked → Blocked/Suspend: swap out to make room for another process.
- Blocked/Suspend → Ready/Suspend: the awaited event occurred while swapped.
- Ready/Suspend → Ready: no ready processes in RAM, or this process has higher priority.
- Ready → Ready/Suspend: only way to free a large enough memory block.
- Running → Ready/Suspend: OS forcibly swaps running process to free memory.
- Any State → Exit: always possible.

■ MEMORIZE — Reasons for Suspension

- Swapping — OS needs memory for a higher-priority ready process.
- OS Reason — suspected of causing a problem (e.g. deadlock).
- Interactive User Request — user pauses/debugs the process.
- Timing — periodic process suspended between executions.
- Parent Process Request — parent suspends child to inspect it.

6. Process Creation & Termination

■ MEMORIZE — Triggers for Process Creation

- Batch job submission.
- User logs on to the system.
- OS creates a process to provide a service (e.g. print spooler).
- Process spawning — a running process (parent) creates another (child).

■ MEMORIZE — Steps of Process Creation (OS actions)

1. Assign a unique PID.
2. Add entry to the primary process table.
3. Allocate memory: program code, data, stack, PCB.
4. Initialize the PCB (state = New, PC = start address, etc.).
5. Set up appropriate linkages (add to Ready or Ready/Suspend queue).
6. Create/expand any other needed data structures (e.g. accounting file).

■ UNDERSTAND — Parent-Child in UNIX (fork/exec)

- fork() — creates a child process as a near-exact copy of the parent. Returns child PID to parent, 0 to child.
- exec() — called by the child after fork() to replace its memory space with a new program.
- wait() — parent blocks until child finishes; collects child's exit status.
- Processes form a tree: init (PID=1) is the root of all processes in Linux.

■ MEMORIZE — Process Termination

- Normal: process calls `exit()` — OS deallocates all resources.
- Parent abort()s a child: child exceeded resources, task no longer needed, or parent is exiting.
- Cascading termination: if a parent dies and OS doesn't allow orphans, all children are also killed.
- Zombie: process finished but parent hasn't called `wait()` yet — PCB still exists.
- Orphan: parent terminated without calling `wait()` — child has no parent.

7. Dispatcher & Context Switch

■ MEMORIZE — Dispatcher

- Small OS program that switches the CPU from one process to another.
- Manages the Ready Queue — each entry is a pointer to a process's PCB.
- Life of a process is bounded by its creation and termination.

■ UNDERSTAND — When Does a Process Switch Happen?

- System Interrupt (external): clock timeout (time slice expired), I/O completion, memory fault.
- Trap (internal): error/exception in the running process (e.g. divide by zero, illegal instruction).
- Supervisor call (system call): process requests OS service → process may be blocked.

■ MEMORIZE — Steps of a Context Switch (process switch)

1. Save context of current process (PC + all registers → PCB).
2. Update PCB: new state, reason for switch, accounting data.
3. Move PCB to appropriate queue (Ready, Blocked, etc.).
4. Select next process to run (scheduling).
5. Update selected process's PCB (state = Running).
6. Update memory management data structures.
7. Restore context of selected process (load PC + registers from PCB).

■ UNDERSTAND — Mode Switch vs Process Switch

- Mode switch: only changes CPU privilege mode (User ↔ Kernel). Much cheaper — only processor state info saved. No process switch needed if the same process continues in kernel mode.
- Process switch: full context save/restore. Required when a different process is to run.

8. OS Control Tables

■ MEMORIZE — Four Types of OS Tables

- Memory Tables: track allocation of main & secondary memory to processes; virtual memory info.
- I/O Tables: which device is free/assigned, status of I/O operation, memory source/destination.
- File Tables: existence and location of files, current status, attributes (managed by File System).
- Process Table: location of each process's image, attributes (PID, state, etc.).

■ UNDERSTAND — Process Image Components

- User Data — modifiable part of user space (program data, user stack area).
- User Program — the executable code to be run.
- System Stack — LIFO stack for each process; stores params and return addresses for procedure/system calls.
- Process Control Block — data needed by OS to control the process (PID, state, registers, memory pointers, etc.).

9. Execution Modes & OS as a Process

■ MEMORIZE — Two Execution Modes

- User Mode — less privileged; where user programs run. Cannot access hardware directly.
- Kernel Mode — more privileged; OS runs here. Can read/write PSW/control registers, do I/O, manage memory.
- Mode is indicated by a flag bit in the PSW (Program Status Word / EFLAGS register).
- Mode changes on: interrupt, trap, or supervisor call (switch to kernel); return from handler (switch to user).

OS Design	Description	Key Point
Non-process Kernel	OS executes outside any process; has its own memory and stack.	Process concept applies only to user programs. Common in older OSes.
Execution within User Processes	OS code runs inside the context of the current user process (shared address space).	Advantage: no process switch needed — just a mode switch when OS routine is called.
Process-based OS	Major OS functions are separate processes running in kernel mode.	Advantages: modular design, clean interfaces, good for multiprocessor systems.

10. UNIX / Linux Process Details

■ MEMORIZE — UNIX Process States (9 states)

- User Running — executing in user mode.
- Kernel Running — executing in kernel mode.
- Ready to Run, in Memory — ready as soon as kernel schedules it.
- Asleep in Memory — blocked, waiting for event; in main memory.
- Ready to Run, Swapped — ready but swapped out to disk.
- Sleeping, Swapped — blocked AND swapped to disk.
- Preempted — returning from kernel to user mode, but kernel preempts it.
- Created — newly forked, not yet ready to run.
- Zombie — process exited; PCB still exists for parent to collect status.

■ MEMORIZE — Linux PCB = `task_struct` (key fields)

- `pid_t pid` — process identifier.
- `long state` — current state of the process.
- `unsigned int time_slice` — scheduling information.
- `struct task_struct *parent` — pointer to parent process.
- `struct list_head children` — list of child processes.
- `struct files_struct *files` — list of open files.
- `struct mm_struct *mm` — address space of this process.
- All active processes → doubly linked list of `task_struct`.
- Kernel pointer `current` → always points to currently executing process.

■ UNDERSTAND — Unix SVR4 Design

- Most OS code executes within the environment of a user process (Execution-within-User-Processes model).
- System Processes run in kernel mode (e.g. memory allocation, process swapping).
- User Processes can switch to kernel mode via system call, exception, or interrupt.

11. Real-World Example — Chrome Multiprocess Architecture

■ UNDERSTAND — Why Multiprocess?

- Old single-process browsers: one bad website crashes the entire browser.
- Chrome uses 3 types of processes for isolation and stability:

Process Type	Responsibility
Browser	Manages UI, disk I/O, network I/O. One per session.
Renderer	Renders web pages (HTML, JS). One per tab/website.
Plug-in	Handles each plug-in type (Flash, QuickTime, etc.).

Advantage: websites run in isolation — if one crashes, only its renderer process is affected.

12. Quick-Review Cheat Sheet

Term	One-Line Definition
Process	A program in execution with its own memory and CPU state.
PCB	Data structure holding all OS info about a process.
Dispatcher	OS routine that switches the CPU between processes.
Context Switch	Save current process state; restore next process state (7 steps).
Mode Switch	Change CPU privilege (user ↔ kernel) — cheaper than process switch.
fork()	UNIX system call that creates a child process (copy of parent).
exec()	Replaces a process's memory space with a new program.
wait()	Parent blocks until child terminates; collects exit status.
Zombie	Process finished but parent hasn't collected its status yet.
Orphan	Process whose parent died without calling wait().
Cascading termination	Killing a parent causes all its children to be killed too.
Swapping	Moving a process out of main memory to disk (suspend state).
Blocked/Suspend	Swapped to disk AND still waiting for an event.
Ready/Suspend	Swapped to disk but ready to run once loaded back.
task_struct	Linux C structure that implements the PCB for each process.
Non-process Kernel	OS runs outside any process in its own memory region.
PSW / EFLAGS	Register containing CPU mode flag and condition codes.

13. OS Requirements w.r.t. Processes

■ **MEMORIZE — What the OS Must Do (all expressed w.r.t. processes)**

- Interleave execution of processes to maximize CPU utilization and provide reasonable response time.
- Allocate resources to processes.
- Protect each process's resources from other processes.
- Enable processes to share and exchange information (IPC — InterProcess Communication).
- Enable synchronization among processes.
- Support creation and termination of processes.